

CAN learning

I learnt CAN as a way to have the devices communicating

There are 3 main abstraction layers: the physical signal; the data-link and transport; the application level.

1.物理信号

CAN接线方式

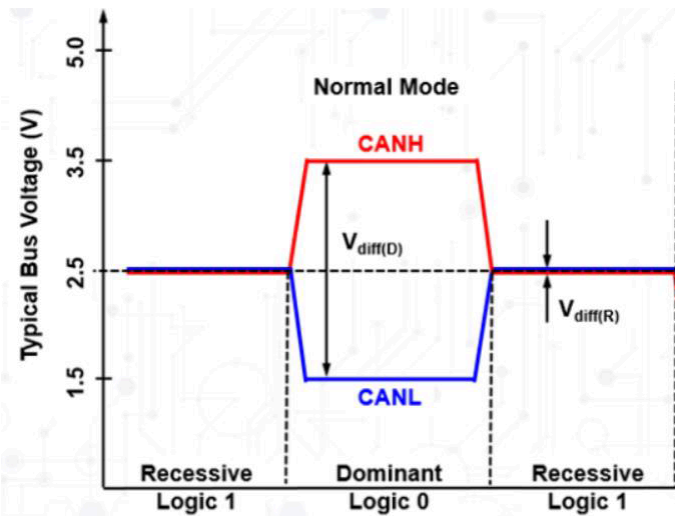
每个设备引出两根线，一根是 CAN_H ，一根是 CAN_L （当然也有单线 CAN 但速度低我们不考虑使用）；不同设备的 CAN_H 接到一起，CAN_L 接到一起CAN_H 与 CAN_L 通过 120Ohm 电阻连接在一起，防止总线上传输的高频信号存在反射振铃效应导致信号不稳定、寄生电容导致信号响应速度变慢等情况（感兴趣可以看电磁场相关的理论）

信号电平格式

差分信号传输： $V_{diff} = V_h - V_l$

0信号-显性- $V_{diff} = 3.5 - 1.5 = 2V$;

1信号-隐性- $V_{diff} = 2.5 - 2.5 = 0V$;



bps = symbol/s 有效信息元就是二进制码元

通过把每个比特（码元）细分时间片，不同时间片做不同的事情,实现异步通信.(1000ns左右发生)

CAN二进制 报文



CAN位时间



同步段调计数器;使得时钟差异被补偿;

传播段等待设备响应;是CAN driver处理时间

缓冲段相位与同步端一起,长度可变,弥补时钟差异的相位差

在两个缓冲段之间的瞬间采样,决定0/1,确保数据准确.

- 概念补充:收发模式

收发模式

- 常规模式

向总线发送,从总线接收

- 回环模式

向总线和本机发送,不从总线接收仅从本机接收

- 静默模式

不向总线发送仅向本机发送,从总线和本机接收

- 回环静默模式

不向总线发送,不从总线接收:自收自发

2.链路传输

几种概念与机制

- CAN ID

一帧信息的标识符,指明是谁发的

在计算机网络中的大多数协议里,地址与一个设备或一个终端完全绑定且一定范畴内一一对应;但 CAN 设备不存在一个固定的地址,而是以 ID 加以区分不同的帧的来源或目标

比如, IP 地址可以在一个网络内唯一指定一个网络设备(不考虑伪造等非法的网络攻击或IP 冲突等错误情况);但 CAN ID 不可以,它只对它发送的报文负责,设备可以发 ID 为 0x201 的报文,也可以发 ID 为 0x202 的报文,每个报文也可以指定不同的信息

- 冲突检测和仲裁

众所周知,一根物理意义的导线上不可能同时是高电平和低电平,一条传统通信方式的链路上也不可能同时传输 0 和 1,因此需要一定的冲突处理机制:

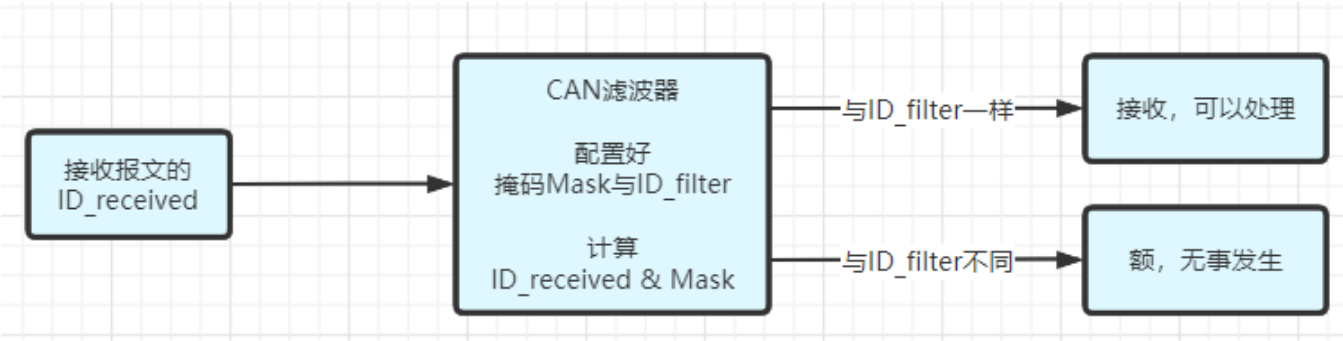
所以,当链路上存在设备想发送 0 的同时又有设备发送 1 的时候,链路中表现为 0,因为 0 是显性电平此时,想发送 1 的设备发现,链路怎么变成 0 了?原来是有人也要发消息,那我沉默先等待.

- 误码效验

差分信号在物理层上一定程度上解决了接口间信道上传输过程中环境噪声问题,而误码校验在链路层上减少了硬件偶然错误的信息编码被解析利用,以及仍可能存在的强干扰环境噪声导致信道电压波动很大改变电平的风险—它采用 CRC algorithm.

- 滤波器与mask code

过滤通过而非排除机制



实现一个用CAN ID和一个Mask 和ID filter 让设备只读特定几种信号

实际情况:帧的处理

在计算机网络上一般而言物理层叫信号，链路层叫帧，网络层叫报文，传输层叫流，再往上叫数据，当然也可以统一都叫数据，我这里把帧这个名字拿来用了

帧的类别:数据帧,遥控帧,错误帧,过载帧,侦间隔

- 数据帧发送数据。细分两种，一种是标准数据帧，一种是扩展数据帧，区别在于它们的CAN ID 长度不同
- 遥控帧请求其他设备给他(发出设备)控制指令(数据帧)，该帧的 ID 表明请求该 ID 对应设备的数据，也有标准遥控帧和扩展遥控帧的两种，区别也是在于它们的 CAN ID 长度不同
- 帧间隔:在前两个主动帧之间发送 错误帧(出错了) 过载帧(忙)

具体内容:

标准帧数据格式			
分段	内容	长度 bit	备注
起始段	SOF, Start Of Frame	1	标志帧起始, 必须为显性电平0
仲裁段	ID, Identifier	11	标识CAN ID, 同时用于仲裁, 显性电平的设备继续发, 隐性电平的设备闭嘴, 换句话说ID大的优先级低
	RTR, Remote Transmission Request	1	远程发送请求, 数据帧中是显性电平0, 遥控帧中是隐性电平1
控制段	IDE, Identifier Extension	1	扩展帧格式标记, 标准数据帧中是显性电平0, 扩展帧中是隐性电平1
	r0	1	保留位, 必须为显性电平0
	DLC, Data Length Code	4	数据长度的字节数, CAN协议中数据长度为0~8字节, 但接收方如果接收到DLC为9以上的数据并不认为出错
数据段	Data	(0~8) * 8	数据内容
校验段	CRC, Cyclic Redundancy Check	15	利用CRC算法生成的校验码
	CRC delimiter	1	CRC界定符, 标识CRC段结束, 必须为隐性电平1
回复段	ACK, ACKnowledge	1	该位由接收设备控制电平, 如果正确接收则为显性电平0, 否则为隐性电平1
	ACK delimiter	1	ACK界定符, 标识ACK段结束, 必须为隐性电平1
结束段	EOF, End Of Frame	7	标志帧结束, 必须为隐性电平1

就关键段说明:

仲裁段就是谁发谁收,确定谁是request

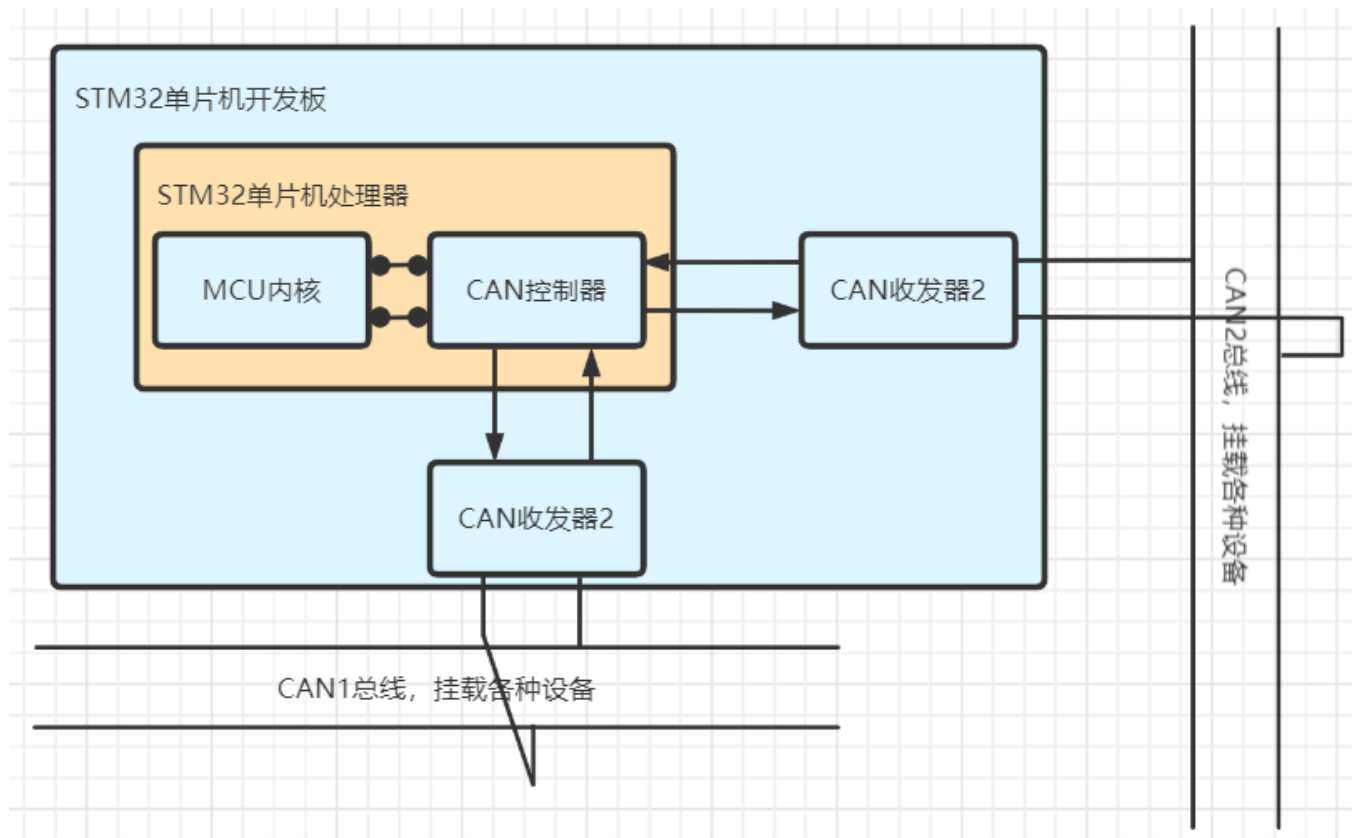
控制段告知data length 和data

效验段用cyclic redundancy check效验

回复段acknowledge 一下是否合作成功

结束段准备下一帧

3.STM32硬件实现应用



发送方式：把发送的数据加到 CAN 收发器中，由它自主控制发送，换句话说放进去就不用管了

接收方式：从接收队列（FIFO 队列）选择接收

应用层配置步骤:

- 元件与引脚对应关系
 - LED1-8 : PG1-PG8 , 电路原理图中低电平点亮
 - CAN1 : PD0、PD1 , 在APB1总线上
- 配置Cube
 - Serial Wire、外部时钟，时钟树不要忘记
 - LED和CAN通信对应引脚配置
 - 工程配置
- 代码编写
 - 初始化配置
 - 发送报文
 - 接收报文

总结:

- 物理信号层，规定了硬件电路的设计规则、用户使用的接线方式、线路上传输信号与电平格式对应关系、波特率比特率、数据同步异步传输、收发模式等内容
- 链路传输层，规定了地址（CANID）、冲突检测与避免（仲裁）、误码校验、滤波器与掩码、该层的帧格式
- 应用数据层是被传输的数据主体；链路层把应用层的数据拿来，加上相关的地址、校验等信息封装；物理层把链路层的数据拿来，编码成物理信号传输出去

附:概念拓展—电机马达

电磁作用化动能

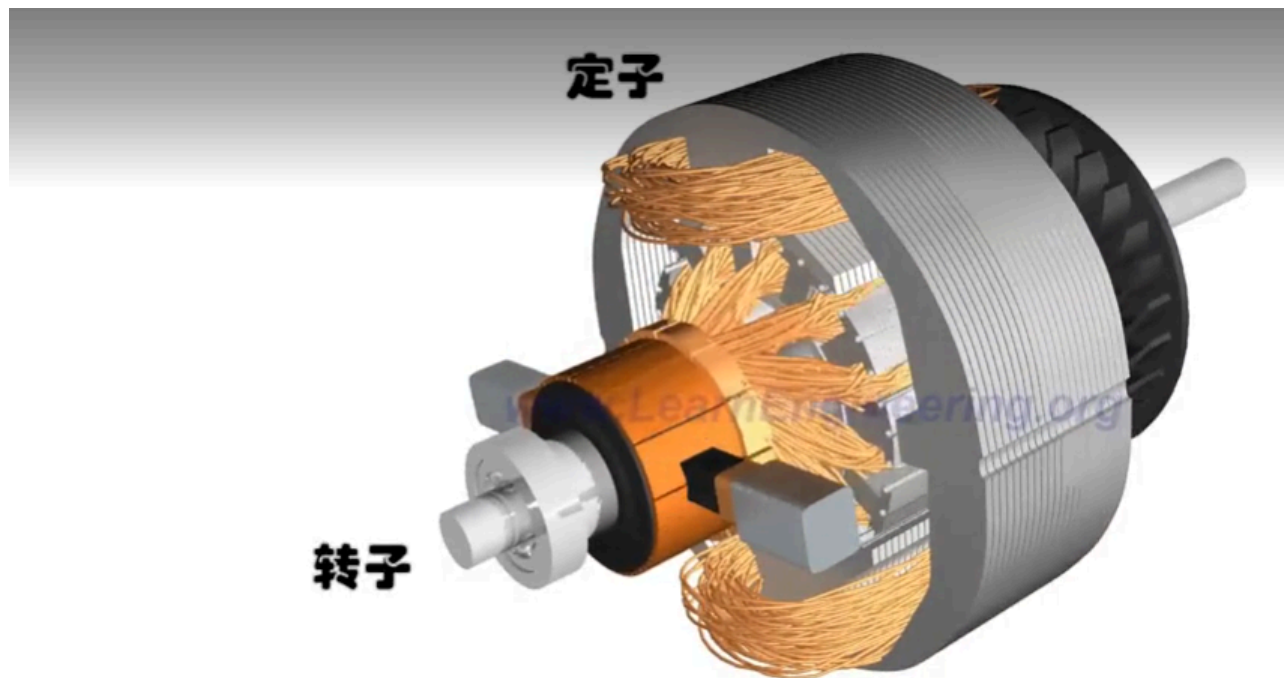
定子+转子

直流-有刷:电流方向-电磁铁是定子(大部分都是他)

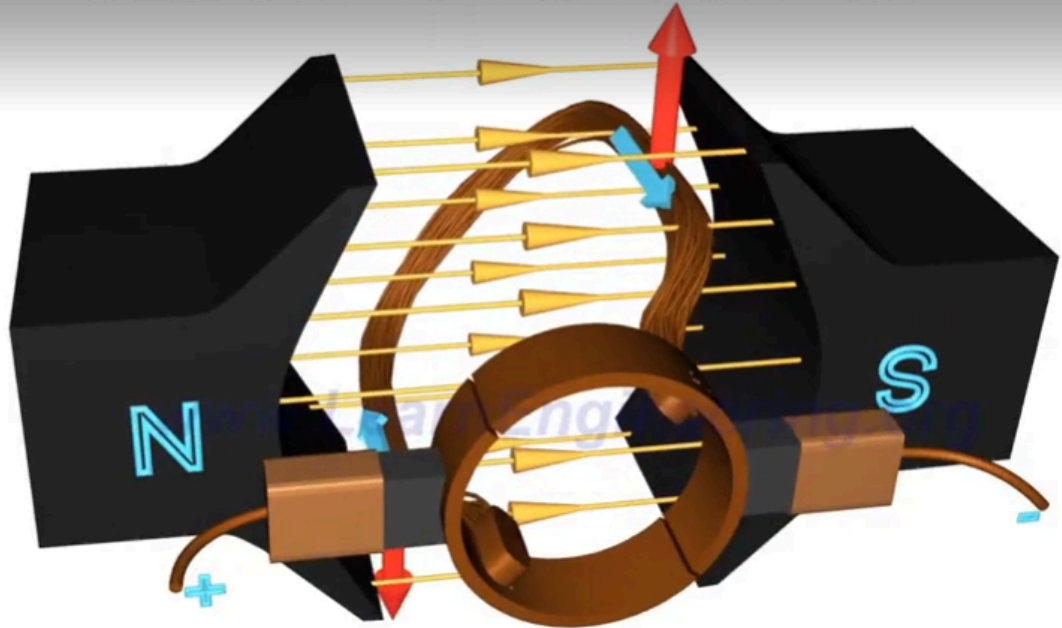
也有直流-无刷

交流-无刷:频率匹配-电磁铁是转子

复习:



线圈通电后，在洛伦兹力的作用下旋转



电机驱动器- 电平格式转换成控制电压电流

用CAN驱动电机:

- 3508电机需要借助C620电调来控制电流，进而实现转动
- CAN通信为1Mbps
- CAN总线上可以允许有多个电调
- 单片机的CAN通信控制电调进而控制电机通过标准数据帧进行
 - 帧的CAN ID为0x200或0x1ff，与电调的CAN ID有关
 - 帧的数据长度为8字节
- 每个电调的CAN ID是不一样的，需要在硬件上进行配置
 - 不同的CAN ID决定了电调获取了单片机发出控制信号对应的数据帧之后，对数据段哪几字节进行处理并转化为电调需要给电机提供的电流
 - 不同的CAN ID决定了电调反馈给单片机的反馈信号对应的数据帧的CAN ID，以便单片机对不同电机运动状态的掌握，以及进一步控制（反馈相关内容在PID应用时讲述）
- 电调的CAN ID不可以相同，否则会有冲突

参考: [【中科大RM电控合集】小白也能看懂的CAN通信+STM32CubeMX编程哔哩哔哩bilibili](#)

电机控制具体CAN信号:

以下是基于 数据段内自定义标识符 的电机控制数据段字节分配示例，结合CAN总线协议和实际应用场景设计:

数据段字节分配（8字节）

字节序号	功能	内容说明	数据格式与示例值
0	目标电调标识符	指定接收指令的电调编号（自定义逻辑，如0x01 = 电调1，0x02 = 电调2）。	0x01 （电调1）

字节序号	功能	内容说明	数据格式与示例值
1-2	目标转速	电机目标转速（16位有符号整数，分辨率=0.5 RPM，范围-32768~32767 RPM）。	0x1F 0x40 → 5000 RPM（解析公式：0x1F40 × 0.5 = 5000）
3	方向控制	正转/反转/停止（1字节枚举值）。	0x01 = 正转，0x02 = 反转，0x00 = 停止
4	使能信号	使能电机驱动器（1字节标志位）。	0x01 = 使能，0x00 = 禁用
5-6	目标电流	电机目标电流（16位有符号整数，分辨率=0.1A，范围-3276.8~3276.8 A）。	0x10 0x00 → 16.0A（解析公式：0x1000 × 0.1 = 16.0）
7	校验和	前7字节的累加和取低8位（用于数据完整性校验）。	0x3A（假设前7字节累加和为0x003A）

完整数据段示例

1 | 数据段（8字节）： [0x01, 0x1F, 0x40, 0x01, 0x01, 0x10, 0x00, 0x3A]

- 第0字节：0x01 → 目标电调1。
- 第1-2字节：0x1F 0x40 → 转速5000 RPM。
- 第3字节：0x01 → 正转。
- 第4字节：0x01 → 使能电机。
- 第5-6字节：0x10 0x00 → 电流16.0A。
- 第7字节：0x3A → 校验和。

关键设计说明

1. 目标电调标识符（第0字节）

- 作用：通过数据段内的自定义标识符指定接收指令的电调，而非依赖CAN仲裁段ID。
- 优势
 - 单片机通过广播发送指令（如CAN ID=0x200），多个电调同时接收。
 - 电调仅处理数据段中标识符匹配自身的帧（如电调1仅响应0x01）。

2. 参数编码规则

- 低字节优先（Little-Endian）：多字节参数按低字节在前传输。
例如：转速5000 RPM → 高字节0x1F（240）先传输，低字节0x40（64）后传输。
- 分辨率定义
 - 转速：0.5 RPM/bit → 16位范围覆盖±32767 RPM。
 - 电流：0.1A/bit → 16位范围覆盖±3276.8A。

3. 校验机制

- **累加和校验**：前7字节累加后取低8位，接收端重新计算并比对。
- **扩展方案**：可替换为CRC-8或CRC-16算法提升可靠性。

实际应用流程

1. **单片机发送控制帧**：
 - **CAN ID**：0x200（广播控制指令）。
 - **数据段**：[0x01, 0x1F, 0x40, 0x01, 0x01, 0x10, 0x00, 0x3A]。
2. **电调1处理流程**：
 - **过滤标识符**：检查数据段第0字节是否为0x01。
 - **解析参数**：提取转速、方向、电流等指令。
 - **执行动作**：驱动电机以5000 RPM正转，电流限制16.0A。
3. **反馈机制**：
 - 电调1通过自身CAN ID（如0x201）发送状态帧，包含实际转速、电流等数据。

通过 **数据段内自定义标识符**（如第0字节）与 **CAN仲裁段ID** 的配合，可实现以下功能：

1. **一对多控制**：单片机广播指令，多个电调通过数据段标识符响应。
2. **灵活扩展**：通过调整标识符和参数编码，支持不同电机或设备。
3. **高可靠性**：校验机制确保数据完整性，避免误操作。

笔记总结

- CAN通信 (Controller Area Network)
 - 我个人认为有三层（这个和真实的标准有出入，但对新手以及不开发硬件的嵌入式开发者来说，理解足够）：物理信号层，链路传输层，应用数据层
 - 物理信号层，规定了硬件电路的设计规则、用户使用的接线方式、线路上传输信号与电平格式对应关系、波特率比特率、数据同步异步传输、收发模式等内容
 - 链路传输层，规定了地址（CAN ID）、冲突检测与避免（仲裁）、误码校验、滤波器与掩码、该层的帧格式
 - 应用数据层是被传输的数据主体；链路层把应用层的数据拿来，加上相关的地址、校验等信息封装；物理层把链路层的数据拿来，编码成物理信号传输出去